

Question 29

Valid File Path Aggregator

You are building a system that processes a large list of file path entries submitted by users. Each entry is a string that is supposed to represent an absolute file path in a UNIX-like system. Your task is to count the total number of unique valid file paths from the given input list.

A valid file path must satisfy all of the following:

- It starts with a / (root directory).
- It contains only valid path components separated by /. Each component:
 - Contains only alphanumeric characters (a-z, A-Z, 0-9), underscores (_), hyphens (-), or dots (.).
 - Is not empty (i.e., no double slashes // anywhere in the path).
- The path must not contain any . or .. components (i.e., no current directory or parent directory references).
- The path must not contain spaces or other invalid characters.

Additionally, trailing slashes at the end of the path should be ignored when determining uniqueness (i.e., /home/user/docs and /home/user/docs/ are considered the same path).

Your program should:

- Parse the input list of file paths.
- Filter out invalid paths.
- Count the total number of unique valid file paths (after normalizing trailing slashes).

If any path is invalid, ignore it and continue processing the rest. Paths are case sensitive.

Function Description:

if any path is invalid, ignore it and continue processing the rest. Paths are case sensitive.

Function Description:

Implement the function `countUniqueValidPaths` that takes a list of strings representing file paths and returns the total count of unique valid paths.

Parameters:

- `N`: An integer represents the number of file paths.
- `paths`: List of strings, each string is a user-submitted file path.

Return:

- An integer representing the number of unique valid file paths.

Input Format:

- The first line contains an integer `N`, the number of file paths.
- The next `N` lines each contain a string representing a file path.

Output Format:

- Return a single integer: the count of unique valid file paths.

Constraints:

- $1 \leq N \leq 10^5$
- Each path length ≤ 300 characters
- Each path component length ≤ 50 characters

Sample Input

7

Sample output

2

[Previous Question](#)

[Next Question](#)

1

```
/home/user/docs  
/home/user/docs/  
/home//user/docs  
/home/user/../docs  
/home/user/.hidden  
/home/user/docs2  
/home/user/docs
```

2

Explanation

- `/home/user/docs` → valid
- `/home/user/docs/` → valid (same as `/home/user/docs` after trailing slash normalization)
- `/home//user/docs` → **invalid** (double slash)
- `/home/user/../docs` → **invalid** (`..` component)
- `/home/user/.hidden` → **invalid** (component starting with dot)
- `/home/user/docs2` → valid (distinct from `/home/user/docs`)
- `/home/user/docs` → duplicate of first valid path

Count = 2

Unique valid paths after normalization:

- `/home/user/docs`
- `/home/user/docs2`

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

[Previous Question](#)

[Next Question](#)

- `/home/user/docs/` → valid (same as `/home/user/docs` after trailing slash normalization)
- `/home//user/docs` → invalid (double slash)
- `/home/user/.docs` → invalid (., component)
- `/home/user/.hidden` → invalid (component starting with dot)
- `/home/user/docs2` → valid (distinct from `/home/user/docs`)
- `/home/user/docs` → duplicate of first valid path

Count = 2

Unique valid paths after normalization:

- `/home/user/docs`
- `/home/user/docs2`

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 5.0 sec(s) for each input file
 Memory Limit: 256 MB
 Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes

Allowed Languages

Bash, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, Java 14, JavaScript(Node.js), Julia, Kotlin, Lisp (SBCL), Lua, Objective-C, OCaml, Octave, Pascal, Perl, PHP, Python, Python 3, Python 3.8, Racket, Ruby, Rust, Scala, Swift, TypeScript, Visual Basic

[← Previous Question](#)


```
1 #include<bits/stdc++.h>
2 using namespace std;
3 int countUniqueValidPaths (int N, vector<vector<string> > paths) {
4     // Write your code here
5 }
6
7 int main() {
8
9     ios::sync_with_stdio(0);
10    cin.tie(0);
11    int N;
12    cin >> N;
13    vector<vector<string> > paths(N, vector<string>(1));
14    for (int i_paths = 0; i_paths < N; i_paths++)
15    {
16        for(int j_paths = 0; j_paths < 1; j_paths++)
17        {
18            cin >> paths[i_paths][j_paths];
19        }
20    }
21
22    int out_;
23    out_ = countUniqueValidPaths(N, paths);
24    cout << out_;
25 }
```

Valid File Path Aggregator

You are building a system that processes a large list of file path entries submitted by users. Each entry is a string that is supposed to represent an absolute file path in a UNIX-like system. Your task is to count the total number of unique valid file paths from the given input list.

A valid file path must satisfy all of the following:

- It starts with a / (root directory).
- It contains only valid path components separated by /. Each component:
 - Contains only alphanumeric characters (a-z, A-Z, 0-9), underscores (_), hyphens (-), or dots (.).
 - Is not empty (i.e., no double slashes // anywhere in the path).
- The path must not contain any . or .. components (i.e., no current directory or parent directory references).
- The path must not contain spaces or other invalid characters.

Additionally, trailing slashes at the end of the path should be ignored when determining uniqueness (i.e., /home/user/docs and /home/user/docs/ are considered the same path).

Your program should:

- Parse the input list of file paths.
- Filter out invalid paths.
- Count the total number of unique valid file paths (after normalizing trailing slashes).

If any path is invalid, ignore it and continue processing the rest. Paths are case sensitive.

Function Description:

Function Description:

Implement the function `countUniqueValidPaths` that takes a list of strings representing file paths and returns the total count of unique valid paths.

Parameters:

- `N`: An integer represents the number of file paths.
- `paths`: List of strings, each string is a user-submitted file path.

Return:

- An integer representing the number of unique valid file paths.

Input Format:

- The first line contains an integer `N`, the number of file paths.
- The next `N` lines each contain a string representing a file path.

Output Format:

- Return a single integer: the count of unique valid file paths.

Constraints:

- $1 \leq N \leq 10^5$
- Each path length ≤ 300 characters
- Each path component length ≤ 50 characters

Sample input

```
7
/home/user/docs
/home/user/docs/
/home/user/docs/
```

Sample output

```
2
```

[Previous Question](#)

[Next Question](#)

Explanation

- `/home/user/docs` → valid
- `/home/user/docs/` → valid (same as `/home/user/docs` after trailing slash normalization)
- `/home//user/docs` → **invalid** (double slash)
- `/home/user/./docs` → **invalid** (`.` component)
- `/home/user/.hidden` → **invalid** (component starting with dot)
- `/home/user/docs2` → valid (distinct from `/home/user/docs`)
- `/home/user/docs` → duplicate of first valid path

Count = 2

Unique valid paths after normalization:

- `/home/user/docs`
- `/home/user/docs2`

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 5.0 sec(s) for each input file
Memory Limit: 256 MB
Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes

Allowed Languages

Bash, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, Java 14, JavaScript(Node.js), Julia, Kotlin, Lisp (SBCL), Lua, Objective-C, OCaml, Octave, Pascal, Perl, PHP, Python, Python 3, Python 3.8, Racket, Ruby, Rust, Scala, Swift, TypeScript, Visual Basic

[← Previous Question](#)

[Next Question →](#)

7

```
/home/user/docs  
/home/user/docs/  
/home//user/docs  
/home/user/../docs  
/home/user/.hidden  
/home/user/docs2  
/home/user/docs
```

2

Explanation

- `/home/user/docs` → valid
- `/home/user/docs/` → valid (same as `/home/user/docs` after trailing slash normalization)
- `/home//user/docs` → **invalid** (double slash)
- `/home/user/../docs` → **invalid** (`..` component)
- `/home/user/.hidden` → **invalid** (component starting with dot)
- `/home/user/docs2` → valid (distinct from `/home/user/docs`)
- `/home/user/docs` → duplicate of first valid path

Count = 2

Unique valid paths after normalization:

- `/home/user/docs`
- `/home/user/docs2`

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

[Previous Question](#)

[Next Question](#)